

BVMT Intelligent Platform

A Beginner's Guide — From Zero to Understanding

For the BI Student Who Feels Lost

Every concept explained with simple words and real examples from things you already know.
Updated edition: Local PostgreSQL (no Docker) + Lightning AI for GPU training.

Esprit School of Business | Academic Year 2025–2026

Before Anything — Read This First

You are a BI student. You know **Excel, SQL, Power BI, dashboards, ETL pipelines, fact tables, dimension tables**. That is **a lot more than you think**. This project is 80% things you already understand, dressed in new vocabulary. This guide removes the scary words and replaces them with things you know.

The honest truth about this project

The BVMT project is basically: collect stock market data (like an ETL pipeline), run some calculations on it (like KPIs in a BI report), feed it into a few AI models (like advanced formula engines), and display the results with explanations. That is it. The fancy words — agents, MCP, GNN, TFT — are just names for pieces of that pipeline.

The Golden Rule for Learning This Project

Every time you see a scary new word, ask: **"What is the BI equivalent of this?"** Almost always, there is one. This guide will always show you the BI equivalent before explaining the new concept.

1. The Big Picture — What Are We Building?

Imagine you are building a **Power BI dashboard for the Tunisian stock market (BVMT)**. But instead of just showing numbers, this dashboard also **predicts what might happen next** and **explains why it thinks that**. That is the whole project.

The 4 Things the Platform Does

#	What It Does	BI Equivalent
1	Collects and organises all BVMT market data in one place	Building a data warehouse from scattered Excel files and PDFs
2	Analyses stock health and patterns	KPI cards and trend charts in a BI dashboard
3	Predicts where a stock price might go next (7 days)	A forecast line on a Power BI chart, but using AI models
4	Explains why it made that prediction in plain language	Analyst commentary and annotations on your dashboard

Who Uses It?

User Type	What They Want	What the Platform Gives Them
Intermediate Investor	Understand which stocks are strong without reading 50-page reports	Health scores, trend direction, risk level — all in one screen
Portfolio Manager	Quickly screen 15 stocks and find sector patterns	Cross-stock analysis, sector comparison, explainable AI predictions

2. The Architecture — Explained Simply

The word "architecture" just means: *how is the project organised and which part talks to which?* Think of it like the **layers in a BI project**: data source → ETL → data warehouse → calculation layer → dashboard. This project has the same idea, just with more layers.

Your BI project: Data Sources → ETL Pipeline → Data Warehouse → KPI Calculations → Dashboard	This project: BVMT Files → DataAgent → PostgreSQL → AI Models → Analysis Result
--	---

The 7 Agents — What Is an Agent?

Simple definition of an Agent

An agent is just a Python class that does ONE specific job. It receives data, does its work, and hands back a result. Think of each agent as one tab in your Excel workbook, or one step in your ETL pipeline.

The only new thing: agents can also make DECISIONS. For example: if there is no news data available, the SentimentAgent skips itself automatically.

Agent	Simple Job	BI Equivalent
DataAgent	Loads stock price CSVs, financial PDFs, and scraped news. Checks data quality.	Your ETL pipeline — Extract + Validate step
TechnicalAgent	Computes RSI, MACD, Bollinger Bands. Runs TFT prediction model.	Calculated columns + forecast formula
FundamentalAgent	Reads company financial ratios (ROE, debt). Gives health score 0–100.	KPI scorecard with red/green flags
SentimentAgent	Reads French and Arabic news. Scores tone as positive or negative.	Sentiment column classifying customer feedback
GraphAgent	Looks at ALL stocks together — models ripple effects between them.	Cross-table correlation analysis, but predictive
RiskAgent	Simulates 10,000 possible price paths. Calculates VaR and CVaR.	Scenario analysis tab — best/worst/average case
XIAgent	Writes plain-language explanation of WHY the model predicted what it did.	Analyst notes section at the bottom of a BI report

3. What Is MCP? (Model Context Protocol)

MCP in one sentence

MCP is a standardised way for an agent to use an external tool (like the database or an AI model) without importing it directly. Instead of writing "import psycpg2" inside your agent code, you just call a tool by name and MCP handles the connection.

What you know from BI:

In Power BI you connect to a database using a connection string. The dashboard does not care HOW the data comes — it just calls the connection and gets results back.

Same idea here:

An agent calls `mcp_client.call_tool("postgres_query", { sql: "..."} )`. The agent does not care HOW the database works — MCP handles the connection.

MCP Server	What Tools It Exposes	Who Uses It
Database Server	Read from PostgreSQL, insert rows, bulk load data	All agents — whenever they need data from the warehouse
Model Server	Run TFT, XGBoost, FinBERT, GNN inference	TechnicalAgent, FundamentalAgent, SentimentAgent, GraphAgent
Files/Web Server	Read CSV files, extract PDF tables, scrape a URL	DataAgent only — at the start of the pipeline

4. The AI Models — No Math, Just Concepts

There are 4 AI models in this project. You do not need to understand their math. You only need to understand **what they eat** (input) and **what they produce** (output).

4.1 TFT — The Price Pattern Reader

BI equivalent:

A trend line or moving average chart in Power BI. It looks at the last 60 days of prices and tries to tell you: is this stock going up or down?

TFT (Temporal Fusion Transformer):

An advanced model that looks at the last 60 trading days + indicators (RSI, MACD) and predicts the MOST LIKELY direction for the next 7 days — with a confidence range.

TFT Input and Output — concrete example for BIAT

INPUT: Last 60 days of Close, Volume, RSI, MACD, Bollinger Bands. Sector = Banking.

OUTPUT:

P10 (pessimistic): -3.2% P50 (most likely): +2.1% P90 (optimistic): +6.8%

Variable importance: RSI 28%, Volume 21%, MACD 19%

4.2 XGBoost — The Financial Health Scorer

XGBoost Output — example for BIAT

Health Class: STRONG | Health Score: 74 / 100

+ ROE = 18% (sector avg 12%) → +20 pts

+ Debt/Equity = 0.42 → low debt: +18 pts

+ Revenue growth +7% → +15 pts

- Net margin declining 15% → 13% → -5 pts

4.3 FinBERT / CAMELBERT — The News Reader

SentimentAgent output — example for BIAT

Sentiment Score: +0.62 | Confidence: 78%

'record profits' → +0.31 'strong growth' → +0.24 'regulatory concern' → -0.18

4.4 GNN — The Network Analyser (The Core Innovation)

The problem GNN solves

All other models look at ONE stock at a time in isolation.

GNN models ALL stocks as a connected network at the same time.

A shock in one stock (BH Bank drops -3.2%) automatically influences related stocks (BIAT). This is the 'ripple effect' — like dropping a stone in water.

5. AgentState — The Shared Notebook

AgentState is just a **shared Python dictionary** that every agent reads from and writes to. Think of it as a **shared Excel sheet** passed from one step to the next in your ETL pipeline.

```
ticker = 'BIAT'           ← which stock are we analysing
user_role = 'manager'     ← who is asking
price_rows_available = 2340 ← DataAgent filled this in
news_articles_count = 8    ← DataAgent: 8 articles found
data_quality_score = 0.94  ← 94% quality
weights = {technical:0.42, ← Orchestrator adjusted these
           fundamental:0.31,
           sentiment:0.27}
skip_sentiment = False     ← enough news, run SentimentAgent
confidence_scores = {technical:0.74, fundamental:0.81, sentiment:0.62}
final_direction = 'UP'     ← filled by GraphAgent
risk_score = 34            ← filled by RiskAgent
```

6. The Data Pipeline — Your Home Territory

The data pipeline here is **exactly like the ETL pipelines you have studied** — Extract from multiple sources, Transform and clean, Load into a structured database.

Table	BI Equivalent	Key Columns
daily_prices	Fact table (transactions)	date, ticker, open, high, low, close, volume
financial_statements	Fact table (events)	ticker, year, quarter, ROE, debt_to_equity, P/E, net_margin
news_articles	Fact table (events)	date, ticker, source, title, body, language
company_metadata	Dimension table	ticker, sector, market_cap_category, listing_date
computed_indicators	Pre-aggregated OLAP cube	date, ticker, RSI, MACD, BB_Upper, ATR (50+ cols)
prediction_results	Fact table (outputs)	date, ticker, direction, prob_up, return_7d
risk_metrics	Fact table (outputs)	date, ticker, VaR_95, CVaR_95, risk_score
audit_log	Audit / compliance table	request_id, ticker, timestamp, result stored for CMF

7. Monte Carlo Risk Engine — Explained Simply

Monte Carlo is basically: **run the same experiment thousands of times with random variation and see what happens most often.**

BI / Excel equivalent: Scenario analysis. You create Best Case, Worst Case, and Base Case columns. Then you calculate portfolio value in each scenario.	Monte Carlo: Instead of 3 scenarios, you simulate 10,000 scenarios. Each one is slightly different. You then count: in how many scenarios does the stock lose money?
---	--

What RiskAgent gives you — plain language

VaR-95 = -2.1% → In 95% of scenarios, loss stays above -2.1%

CVaR-95 = -3.2% → In the worst 5% of scenarios, average loss is -3.2%

Risk Score = 34 / 100 → Moderate risk. More than half of scenarios end positively.

Probability of any loss = 34.2%

8. XAI — Making the AI Explain Itself

XAI (Explainable AI) just means: **the system must explain its reasoning in plain language.**

Final XAI output paragraph — shown to the user

"Prediction: UPWARD (+2.8%, confidence: 71%) for BIAT Bank over 7 days."

Why this prediction:

1. Technical momentum is BULLISH: RSI=61, MACD positive, volume confirming (42% of weight)
2. Financials are SOLID: ROE 18% vs sector avg 12%, D/E = 0.42 (31% of weight)
3. Sentiment is POSITIVE: 'record profits', 'strong growth' scored +0.62 (27% of weight)

Sector influence: BH Bank drop adds -0.8% drag. Risk Score: 34/100.

This is an educational tool. Not investment advice.

9. Sprint 1 — Full Step-by-Step Guide (Days 1–10)

Your setup: Local PostgreSQL + pgAdmin (already installed) + Lightning AI for GPU training

No Docker needed. Your PostgreSQL is already running on Windows.

Lightning AI (lightning.ai) replaces Google Colab for training heavy models — free tier available.

Everything else runs locally on your Windows machine.

9.0 Setting Up Lightning AI (Do This Before Day 1)

Lightning AI is a free cloud platform that gives you a GPU-powered environment for training your heavy AI models (TFT, GNN). Think of it as **a powerful cloud computer you access in your browser** — you upload your data and training scripts, run them on a free GPU, then download the trained model back to your machine.

How to Set Up Lightning AI — 5 Steps

1. Go to lightning.ai in your browser. Click "Sign Up Free". Use your Google or GitHub account.
2. After signing in, click "New Studio". Choose "Blank Studio". Name it "bvmt-training".
3. In the Studio, you get a Jupyter-like environment with a free GPU (T4). You see a file browser on the left.
4. To upload your training data: drag CSV files from your computer into the Studio file browser — or use the upload button.
5. To run a training script: create a .py file in the Studio, paste your training code, then click Run or open a terminal and type: `python train_tft.py`

Lightning AI vs Google Colab — key differences

Lightning AI: persistent storage (files stay between sessions), better GPU availability, more professional workflow.

Google Colab: files disappear when session ends, you had to re-upload data every time.

For this project: upload your CSV data files once to Lightning AI Studio — they stay there permanently.

Free tier gives you enough GPU hours to train TFT and GNN comfortably.

9.1 Project Folder Structure (Create This on Day 1)

Open **VS Code**. Open a folder (File → Open Folder). Create a new empty folder called **bvmt_project**. Then create the following structure inside it:

```
bvmt_project/
├── agents/           ← Python classes for each agent
│   ├── __init__.py  ← makes this folder a Python package (empty file)
│   └── agent_state.py ← the shared AgentState dataclass
```

```
|   |   | orchestrator.py      ← the OrchestratorAgent
|   |   | data_agent.py      ← the DataAgent
|   |   | base_agent.py      ← base class all agents inherit from
|   |   |
|   |   | mcp_servers/        ← MCP server files
|   |   |   | database_server.py ← exposes PostgreSQL tools
|   |   |   | files_server.py   ← exposes file/web tools
|   |   |
|   |   | models/            ← trained model files (.pt, .pkl)
|   |   |   | .gitkeep         ← empty file to keep folder in git
|   |   |
|   |   | data/
|   |   |   | prices/          ← BVMT price CSV files go here
|   |   |   | financials/      ← PDF annual reports go here
|   |   |   | news/            ← news CSV files go here
|   |   |
|   |   | tests/             ← unit tests
|   |   |   | test_data_agent.py
|   |   |
|   |   | training/          ← scripts you upload to Lightning AI
|   |   |   | train_tft.py     ← TFT training script (Week 3)
|   |   |
|   |   | sql/
|   |   |   | schema.sql       ← database creation script
|   |   |
|   |   | requirements.txt    ← list of Python packages
|   |   |
|   |   | main.py            ← entry point (Week 11)
```

How to create files and folders quickly in VS Code

In VS Code Explorer panel (left side): right-click → New Folder to create a folder.
Right-click → New File to create a file.
To create __init__.py: right-click the agents folder → New File → type __init__.py → press Enter → leave it empty.

9.2 Day-by-Day Guide

WEEK 1 (Days 1–5) — Environment + Database + First Data	Tasks
Day 1 Mon	• Install Python 3.11 from python.org (tick "Add to PATH"). Install VS Code from code.visualstudio.com. Install Git from git-scm.com. • Open VS Code. Install the Python extension (Extensions panel → search "Python" → Install). • Open a VS Code terminal (Terminal → New Terminal). Create your project folder: mkdir bvmt_project && cd bvmt_project • Create Python virtual environment: python -m venv venv • Activate the virtual environment: venv\Scripts\activate (you should see (venv) in front of your terminal prompt) • Create requirements.txt file with this content: pandas, pdfplumber, requests, beautifulsoup4, psycopg2-binary, asyncpg, mcp, pandas-ta, python-dotenv • Install everything: pip install -r requirements.txt

WEEK 1 (Days 1–5) — Environment + Database + First Data	Tasks
	• Create all folders from the tree above. Create empty <code>__init__.py</code> files in <code>agents/</code> and <code>mcp_servers/</code>. • Verify: In terminal type <code>python -c "import pandas; print(pandas.__version__)"</code> — you should see a version number like 2.x.x
Day 2 Tue	• Open pgAdmin (you already have it installed). Right-click "Databases" → Create → Database. Name it: <code>bvmt_db</code>. Click Save. • In pgAdmin: right-click <code>bvmt_db</code> → Query Tool. This opens a SQL editor — just like SSMS or DBeaver. • Create the file <code>sql/schema.sql</code> with the complete schema (see full code below). Then paste it into pgAdmin Query Tool and press F5 to run. • Verify: in pgAdmin expand <code>bvmt_db</code> → Schemas → public → Tables. You should see 11 tables listed. • Create a <code>.env</code> file in your project root with: <code>DB_HOST=localhost, DB_PORT=5432, DB_NAME=bvmt_db, DB_USER=postgres, DB_PASSWORD=your_password</code> • Create <code>agents/agent_state.py</code> with the <code>AgentState</code> dataclass (see full code below). • Test <code>AgentState</code>: <code>python -c "from agents.agent_state import AgentState; s=AgentState(ticker='BIAT',user_role='manager'); print(s.ticker)"</code> — should print <code>BIAT</code>
Day 3 Wed	• Go to <code>bvmt.com.tn</code>. Navigate to each stock (<code>BIAT</code>, <code>BH Bank</code>, <code>Attijari Bank</code>, <code>SFBT</code>, <code>Tunisie Telecom</code>). Download the historical price data as CSV. • Save each file as: <code>data/prices/BIAT.csv</code>, <code>data/prices/BH.csv</code>, <code>data/prices/ATB.csv</code>, <code>data/prices/SFBT.csv</code>, <code>data/prices/TT.csv</code> • Open one CSV in Excel first to check the column names. The most common format is: Date, Open, High, Low, Close, Volume. Rename columns if needed. • Create <code>tests/test_data_load.py</code> (see code below) and run it: <code>python tests/test_data_load.py</code> — you should see row counts for each stock. • Create <code>mcp_servers/database_server.py</code> (see full code below). • Test the database server: <code>python mcp_servers/database_server.py</code> (it should start without errors — press <code>Ctrl+C</code> to stop).
Day 4 Thu	• Create <code>agents/data_agent.py</code> — the <code>DataAgent</code> class that loads CSV files and checks quality (see full code below). • Run the <code>DataAgent</code> test: <code>python -c "import asyncio; from agents.data_agent import DataAgent; from agents.agent_state import AgentState; s=AgentState(ticker='BIAT',user_role='manager'); asyncio.run(DataAgent().run(s))"</code> — should print a <code>DataReport</code>. • Load all 5 CSVs into PostgreSQL using the <code>DataAgent</code>: create and run a script called <code>load_initial_data.py</code> (see code below). • Verify in pgAdmin: run <code>SELECT ticker, COUNT(*) FROM daily_prices GROUP BY ticker</code> — you should see 5 rows with row counts.
Day 5 Fri	• Download CMF annual report PDFs for at least 2 stocks (<code>BIAT</code> and <code>BH Bank</code>). Save in <code>data/financials/</code>. • Run <code>pdfplumber</code> on one PDF to test extraction: create <code>test_pdf.py</code> (see code below). Run: <code>python test_pdf.py</code>

WEEK 1 (Days 1–5) — Environment + Database + First Data	Tasks
	• Collect news articles: go to leconomiste.com.tn and search for "BIAT bourse". Copy 5–10 article titles and text into <code>data/news/BIAT_news.csv</code> manually. • Create a CSV with columns: date, ticker, source, title, body, language. Fill in a few rows by hand for now. • Buffer and review: read back through everything you built this week. Make sure all 5 stocks are in the database and you can query them in pgAdmin. • Checkpoint: you should be able to run <code>python -c "from agents.data_agent import DataAgent"</code> without any import errors.
WEEK 2 (Days 6–10) — MCP Servers + Orchestrator + Web Scraping	Tasks
Day 6 Mon	• Create <code>agents/base_agent.py</code> — a simple base class all agents will inherit from (see code below). • Create <code>agents/orchestrator.py</code> — the <code>OrchestratorAgent</code> with skeleton methods: <code>__init__</code>, <code>run()</code>, <code>_adjust_weights()</code>, <code>_empty_output()</code> (see full code below). • Test weight adjustment: create a quick test file that creates an <code>AgentState</code> with <code>news_articles_count=2</code> and calls <code>_adjust_weights()</code>. Print the weights before and after. They should change. • Verify: after adjustment with only 2 articles, sentiment weight should decrease from 0.27 and technical weight should increase.
Day 7 Tue	• Create <code>mcp_servers/files_server.py</code> — MCP server that exposes <code>read_csv</code>, <code>read_pdf_tables</code>, and <code>fetch_url</code> tools (see full code below). • Test files server: <code>python mcp_servers/files_server.py</code> (should start without errors). • Add web scraping to <code>DataAgent</code>: implement the <code>scrape_news()</code> method using <code>BeautifulSoup4</code> to scrape headlines from bvm.com.tn announcements page. • Test scraping: <code>python -c "from agents.data_agent import DataAgent; import asyncio; asyncio.run(DataAgent().scrape_news('BIAT'))"</code>
Day 8 Wed	• Expand data collection: download PDFs for all 5 stocks and run <code>pdfplumber</code> extraction on all of them. • Implement <code>DataAgent.ingest_pdf()</code> — reads a PDF file, extracts tables, looks for ROE / D/E / P/E values, loads them into <code>financial_statements</code> table. • Run ingestion for all 5 stocks: <code>python -c "from agents.data_agent import DataAgent; import asyncio; asyncio.run(DataAgent().ingest_all_financials())"</code> • Verify in pgAdmin: <code>SELECT ticker, year, roe, debt_to_equity FROM financial_statements ORDER BY ticker, year</code> — should see data for 5 stocks.
Day 9 Thu	• Implement correlation matrix computation: add a <code>compute_correlations()</code> method to <code>DataAgent</code>.

WEEK 2 (Days 6–10) — MCP Servers + Orchestrator + Web Scraping	Tasks
	• Load closing prices for all 5 stocks, compute pandas .corr(), store each pair in correlation_matrix table. • Run: <code>python -c "from agents.data_agent import DataAgent; import asyncio; asyncio.run(DataAgent().compute_correlations())"</code> • Verify: <code>SELECT ticker_a, ticker_b, pearson_corr FROM correlation_matrix</code> — you should see 10 pairs (5×5 unique combinations). • Implement <code>DataAgent.compute_data_quality_score()</code> — counts missing rows, zero-volume days, returns a float 0.0 to 1.0.
Day 10 Fri	• Full Sprint 1 integration test: create <code>tests/test_sprint1.py</code> that runs: <code>DataAgent</code> for all 5 stocks → checks database has data → <code>Orchestrator</code> initialises <code>AgentState</code> → <code>adjust_weights()</code> runs correctly. • Run the test: <code>python tests/test_sprint1.py</code> — all assertions should pass. • Open pgAdmin and verify: 5 rows in <code>company_metadata</code>, 5 distinct tickers in <code>daily_prices</code>, at least some rows in <code>financial_statements</code> and <code>correlation_matrix</code>. • Write a short <code>sprint_1_notes.txt</code>: what was easy, what was hard, what took longer than expected. • Lightning AI prep: go to lightning.ai, create your Studio if not done yet. Upload your CSV files from <code>data/prices/</code> to the Studio. You will need them in Sprint 2 for TFT training.

9.3 Complete Code Files for Sprint 1

Below are the complete code files. Copy them exactly — do not skip anything. The comments explain every line.

File: `sql/schema.sql`

```
-- BVMT Data Warehouse Schema
-- Run this in pgAdmin Query Tool (F5 to execute)

-- 1. Daily stock prices (Fact Table)
CREATE TABLE IF NOT EXISTS daily_prices (
  id          SERIAL PRIMARY KEY,
  date        DATE NOT NULL,
  ticker       VARCHAR(20) NOT NULL,
  open        NUMERIC(10,4),
  high        NUMERIC(10,4),
  low         NUMERIC(10,4),
  close       NUMERIC(10,4) NOT NULL,
  volume      BIGINT,
  created_at  TIMESTAMP DEFAULT NOW(),
  UNIQUE(date, ticker)
);
```

```

-- 2. Financial statements (Fact Table)
CREATE TABLE IF NOT EXISTS financial_statements (
    id SERIAL PRIMARY KEY,
    ticker VARCHAR(20) NOT NULL,
    year INTEGER NOT NULL,
    quarter INTEGER DEFAULT 4,
    roe NUMERIC(8,4), -- Return on Equity
    roa NUMERIC(8,4), -- Return on Assets
    debt_to_equity NUMERIC(8,4),
    pe_ratio NUMERIC(10,4),
    net_margin NUMERIC(8,4),
    revenue_growth NUMERIC(8,4),
    current_ratio NUMERIC(8,4),
    created_at TIMESTAMP DEFAULT NOW(),
    UNIQUE(ticker, year, quarter)
);

-- 3. News articles (Fact Table)
CREATE TABLE IF NOT EXISTS news_articles (
    id SERIAL PRIMARY KEY,
    date DATE,
    ticker VARCHAR(20),
    source VARCHAR(100),
    title TEXT,
    body TEXT,
    language VARCHAR(5), -- "fr" or "ar"
    source_credibility NUMERIC(3,2) DEFAULT 0.7,
    sentiment_score NUMERIC(4,3),
    created_at TIMESTAMP DEFAULT NOW()
);

-- 4. Company metadata (Dimension Table)
CREATE TABLE IF NOT EXISTS company_metadata (
    ticker VARCHAR(20) PRIMARY KEY,
    name VARCHAR(200),
    sector VARCHAR(100),
    market_cap_category VARCHAR(20), -- Large, Mid, Small
    listing_date DATE,
    isin VARCHAR(20)
);

-- 5. Computed indicators (Pre-aggregated OLAP cube)
CREATE TABLE IF NOT EXISTS computed_indicators (
    id SERIAL PRIMARY KEY,
    date DATE NOT NULL,
    ticker VARCHAR(20) NOT NULL,
    rsi NUMERIC(8,4),
    macd NUMERIC(10,6),
    bb_upper NUMERIC(10,4),
    bb_lower NUMERIC(10,4),
    atr NUMERIC(10,4),
    obv BIGINT,
    UNIQUE(date, ticker)
);

```

```

-- 6. Correlation matrix
CREATE TABLE IF NOT EXISTS correlation_matrix (
    id          SERIAL PRIMARY KEY,
    ticker_a    VARCHAR(20),
    ticker_b    VARCHAR(20),
    pearson_corr NUMERIC(6,4),
    computed_date DATE DEFAULT CURRENT_DATE,
    UNIQUE(ticker_a, ticker_b)
);

-- 7. Feature vectors (Feature Store)
CREATE TABLE IF NOT EXISTS feature_vectors (
    id          SERIAL PRIMARY KEY,
    date        DATE,
    ticker      VARCHAR(20),
    vector_json TEXT, -- 88-dim vector as JSON string
    weights_used TEXT,
    UNIQUE(date, ticker)
);

-- 8. Prediction results
CREATE TABLE IF NOT EXISTS prediction_results (
    id          SERIAL PRIMARY KEY,
    date        DATE,
    ticker      VARCHAR(20),
    direction   VARCHAR(10), -- UP, DOWN, NEUTRAL
    prob_up     NUMERIC(5,4),
    return_7d   NUMERIC(8,4),
    confidence  NUMERIC(5,4),
    model_used  VARCHAR(50)
);

-- 9. Risk metrics
CREATE TABLE IF NOT EXISTS risk_metrics (
    id          SERIAL PRIMARY KEY,
    date        DATE,
    ticker      VARCHAR(20),
    var_95      NUMERIC(8,4),
    var_99      NUMERIC(8,4),
    cvar_95     NUMERIC(8,4),
    risk_score  INTEGER,
    fan_chart   TEXT
);

-- 10. CMF Audit log
CREATE TABLE IF NOT EXISTS audit_log (
    id          SERIAL PRIMARY KEY,
    request_id  VARCHAR(50),
    ticker      VARCHAR(20),
    user_role   VARCHAR(20),
    timestamp   TIMESTAMP DEFAULT NOW(),
    result_json TEXT
);

```

```
-- 11. Knowledge Hub posts
CREATE TABLE IF NOT EXISTS hub_posts (
    id SERIAL PRIMARY KEY,
    author_id INTEGER,
    ticker VARCHAR(20),
    title VARCHAR(300),
    body TEXT,
    moderation_status VARCHAR(20) DEFAULT 'pending',
    created_at TIMESTAMP DEFAULT NOW()
);

-- Verify: SELECT table_name FROM information_schema.tables WHERE
table_schema='public';
-- You should see 11 tables listed.
```

File: agents/agent_state.py

```
# agents/agent_state.py
# The shared "notebook" passed between all agents.
# Think of it as a config dict that every ETL step reads and updates.

from dataclasses import dataclass, field
from typing import Optional, Dict, Any, List
from datetime import datetime
import uuid

@dataclass
class AgentState:
    # --- Who are we analysing? ---
    ticker: str # e.g. "BIAT"
    user_role: str # "intermediate" or "manager"
    request_id: str = field(default_factory=lambda: str(uuid.uuid4()))
    created_at: datetime = field(default_factory=datetime.utcnow)

    # --- Filled by DataAgent (quality report) ---
    price_rows_available: int = 0
    news_articles_count: int = 0
    financial_statements_count: int = 0
    data_quality_score: float = 0.0
    has_recent_financials: bool = False

    # --- Dynamic weights (Orchestrator adjusts these) ---
    # Default split: 42% technical, 31% fundamental, 27% sentiment
    weights: Dict[str, float] = field(default_factory=lambda: {
        "technical": 0.42,
        "fundamental": 0.31,
        "sentiment": 0.27
    })

    # --- Routing flags (Orchestrator sets these) ---
    skip_sentiment: bool = False # True if < 5 news articles
    skip_fundamental: bool = False # True if no financial data
    use_cnn_lstm_fallback: bool = False # True if TFT crashes
```

```

# --- Results from each agent ---
confidence_scores: Dict[str, float] = field(default_factory=dict)
agent_outputs: Dict[str, Any] = field(default_factory=dict)
attention_weights: Dict[str, float] = field(default_factory=dict)
sector_shock_detected: bool = False

# --- Final assembled results ---
final_direction: Optional[str] = None # "UP", "DOWN", "NEUTRAL"
final_return_7d: Optional[float] = None
risk_score: Optional[int] = None
overall_confidence: float = 0.0

# --- Audit trail ---
execution_log: List[str] = field(default_factory=list)

def log(self, message: str):
    """Add a timestamped message to the execution log."""
    self.execution_log.append(f"[{datetime.utcnow().isoformat()}]
{message}")
    print(message) # also print to console so you can see progress

```

File: agents/base_agent.py

```

# agents/base_agent.py
# Base class for all agents. Every agent inherits from this.
# Provides shared database connection and a standard output format.

import os
import asyncpg
from dataclasses import dataclass, field
from typing import Any, List, Optional
from dotenv import load_dotenv

load_dotenv() # reads your .env file

@dataclass
class AgentOutput:
    """Standard format every agent must return."""
    agent_name: str
    ticker: str
    status: str # "success", "partial", "failed"
    confidence: float # 0.0 to 1.0
    payload: dict = field(default_factory=dict)
    warnings: List[str] = field(default_factory=list)
    error: Optional[str] = None

class BaseAgent:
    """All agents inherit from this class."""

    def __init__(self):
        self.db_config = {

```

```

        "host": os.getenv("DB_HOST", "localhost"),
        "port": int(os.getenv("DB_PORT", 5432)),
        "database": os.getenv("DB_NAME", "bvmt_db"),
        "user": os.getenv("DB_USER", "postgres"),
        "password": os.getenv("DB_PASSWORD", ""),
    }

    async def get_db(self):
        """Get a database connection. Use in agent methods."""
        return await asyncpg.connect(**self.db_config)

    async def query_db(self, sql: str, *params) -> list:
        """Run a SELECT query and return list of rows."""
        conn = await self.get_db()
        try:
            rows = await conn.fetch(sql, *params)
            return [dict(r) for r in rows]
        finally:
            await conn.close() # always close the connection

    async def execute_db(self, sql: str, *params):
        """Run an INSERT/UPDATE query."""
        conn = await self.get_db()
        try:
            await conn.execute(sql, *params)
        finally:
            await conn.close()

```

File: agents/data_agent.py

```

# agents/data_agent.py
# The DataAgent – first agent called by Orchestrator.
# Loads CSV prices, PDF financials, and news. Checks quality.

import os
import pandas as pd
import pdfplumber
from datetime import datetime, timedelta
from agents.base_agent import BaseAgent, AgentOutput
from agents.agent_state import AgentState

class DataAgent(BaseAgent):

    async def run(self, state: AgentState) -> AgentOutput:
        """Main entry point called by Orchestrator."""
        state.log(f"DataAgent: starting for {state.ticker}")
        try:
            price_count = await self.load_prices(state.ticker)
            news_count = await self.load_news(state.ticker)
            fin_count = await self.load_financials(state.ticker)
            quality = self._compute_quality(price_count, news_count,
            fin_count)

```

```

        # Fill AgentState with what we found
        state.price_rows_available      = price_count
        state.news_articles_count      = news_count
        state.financial_statements_count = fin_count
        state.data_quality_score        = quality
        state.has_recent_financials     = fin_count > 0

        state.log(f"DataAgent: {price_count} price rows, {news_count}
articles, quality={quality:.2f}")
        return AgentOutput(agent_name="DataAgent", ticker=state.ticker,
                           status="success", confidence=quality,
payload={"price_rows":price_count,"news":news_count,"financials":fin_count})
    except Exception as e:
        state.log(f"DataAgent ERROR: {e}")
        return AgentOutput(agent_name="DataAgent", ticker=state.ticker,
                           status="failed", confidence=0.0, error=str(e))

    async def load_prices(self, ticker: str) -> int:
        """Load a CSV price file and insert rows into daily_prices table."""
        csv_path = f"data/prices/{ticker}.csv"
        if not os.path.exists(csv_path):
            return 0

        df = pd.read_csv(csv_path)

        # Normalise column names - handle different BVMt CSV formats
        df.columns = [c.strip().lower().replace(" ", "_") for c in df.columns]
        if 'date' not in df.columns: return 0

        df["date"] = pd.to_datetime(df["date"], dayfirst=True) # BVMt uses
DD/MM/YYYY
        df = df.dropna(subset=["close"]) # remove rows with no closing
price
        df = df[df.get("volume", pd.Series([1]*len(df))) > 0] # remove
market-closed days
        df["ticker"] = ticker

        # Insert into database (skip duplicates with ON CONFLICT DO NOTHING)
        conn = await self.get_db()
        inserted = 0
        try:
            for _, row in df.iterrows():
                await conn.execute(
                    """INSERT INTO daily_prices
(date,ticker,open,high,low,close,volume)
VALUES ($1,$2,$3,$4,$5,$6,$7)
ON CONFLICT (date,ticker) DO NOTHING""",
                    row["date"].date(), ticker,
                    float(row.get("open",0) or 0),
                    float(row.get("high",0) or 0),
                    float(row.get("low",0) or 0),
                    float(row["close"]),
                    int(row.get("volume",0) or 0)
                )

```

```

        inserted += 1
    finally:
        await conn.close()
    return inserted

async def load_news(self, ticker: str) -> int:
    """Load a news CSV file into news_articles table."""
    csv_path = f"data/news/{ticker}_news.csv"
    if not os.path.exists(csv_path):
        return 0
    df = pd.read_csv(csv_path)
    conn = await self.get_db()
    count = 0
    try:
        for _, row in df.iterrows():
            await conn.execute(
                """INSERT INTO news_articles
(date,ticker,source,title,body,language)
VALUES ($1,$2,$3,$4,$5,$6)
ON CONFLICT DO NOTHING""",
                pd.to_datetime(row.get("date")).date() if
pd.notna(row.get("date")) else None,
                ticker, str(row.get("source","")),
                str(row.get("title","")), str(row.get("body","")),
                str(row.get("language","fr"))
            )
            count += 1
    finally:
        await conn.close()
    return count

async def load_financials(self, ticker: str) -> int:
    """Extract financial ratios from a PDF annual report."""
    pdf_path = f"data/financials/{ticker}_annual.pdf"
    if not os.path.exists(pdf_path):
        return 0
    data = {}
    with pdfplumber.open(pdf_path) as pdf:
        for page in pdf.pages:
            for table in (page.extract_tables() or []):
                for row in table:
                    row = [str(c).upper() if c else "" for c in row]
                    if "ROE" in str(row):
                        for i, cell in enumerate(row):
                            if "ROE" in cell and i+1 < len(row):
                                try: data["roe"] =
float(row[i+1].replace(",","."))
                                except: pass

    if not data:
        return 0
    await self.execute_db(
        """INSERT INTO financial_statements (ticker,year,roe)
VALUES ($1,$2,$3)
ON CONFLICT (ticker,year,quarter) DO NOTHING""",
        ticker, datetime.now().year - 1, data.get("roe"))
    return 1

```

```

def _compute_quality(self, prices, news, financials) -> float:
    """Score 0.0 to 1.0 based on how much data is available."""
    score = 0.0
    if prices > 500:    score += 0.50 # at least 2 years of prices
    elif prices > 100:  score += 0.25
    if news > 10:       score += 0.25
    elif news > 0:      score += 0.10
    if financials > 0:  score += 0.25
    return round(min(score, 1.0), 2)

async def compute_correlations(self):
    """Compute Pearson correlations between all stock pairs."""
    rows = await self.query_db("SELECT date, ticker, close FROM
daily_prices ORDER BY ticker, date")
    if not rows: return
    df = pd.DataFrame(rows).pivot(index="date", columns="ticker",
values="close")
    corr = df.corr()
    conn = await self.get_db()
    try:
        for t1 in corr.columns:
            for t2 in corr.columns:
                if t1 >= t2: continue
                val = corr.loc[t1, t2]
                if pd.isna(val): continue
                await conn.execute(
                    """INSERT INTO correlation_matrix
(ticker_a,ticker_b,pearson_corr)
VALUES ($1,$2,$3) ON CONFLICT (ticker_a,ticker_b)
DO UPDATE SET
pearson_corr=EXCLUDED.pearson_corr""",
                    t1, t2, float(val))
    finally:
        await conn.close()
    print(f"Correlations computed for {len(corr.columns)} stocks")

```

File: agents/orchestrator.py

```

# agents/orchestrator.py
# The OrchestratorAgent – the manager that decides which agents run.
# Sprint 1: skeleton only. More logic added in Sprints 2-5.

from agents.agent_state import AgentState
from agents.data_agent import DataAgent

class OrchestratorAgent:

    def __init__(self):
        self.data_agent = DataAgent()
        # Other agents added in later sprints:
        # self.technical_agent = TechnicalAgent() # Sprint 2
        # self.fundamental_agent = FundamentalAgent() # Sprint 3

```

```

# self.sentiment_agent = SentimentAgent() # Sprint 3
# self.graph_agent = GraphAgent() # Sprint 4

async def run(self, ticker: str, user_role: str) -> dict:
    """Main entry point. Called from FastAPI in Sprint 6."""
    state = AgentState(ticker=ticker, user_role=user_role)
    state.log(f"Orchestrator: starting analysis for {ticker}")

    # Step 1: Data quality check (always first)
    data_output = await self.data_agent.run(state)
    state.agent_outputs["data"] = data_output

    # Step 2: Adjust weights based on data quality
    state = self._adjust_weights(state)

    state.log(f"Orchestrator: weights after adjustment = {state.weights}")

    # Steps 3-7 added in later sprints
    # For now, return what we have
    return {
        "ticker": ticker,
        "data_quality": state.data_quality_score,
        "weights": state.weights,
        "price_rows": state.price_rows_available,
        "news_count": state.news_articles_count,
        "status": "sprint_1_complete"
    }

def _adjust_weights(self, state: AgentState) -> AgentState:
    """Rebalance weights based on what data is actually available."""
    w = dict(state.weights)

    # If very few news articles, reduce sentiment weight
    if state.news_articles_count < 5:
        deficit = w["sentiment"] * 0.7 # remove 70% of sentiment weight
        w["sentiment"] -= deficit
        w["technical"] += deficit * 0.6 # give more to technical
        w["fundamental"] += deficit * 0.4
        state.skip_sentiment = True
        state.log("Orchestrator: low news count → skip sentiment, boost
technical")

    # If no financial statements, reduce fundamental weight
    if not state.has_recent_financials:
        deficit = w["fundamental"] * 0.5
        w["fundamental"] -= deficit
        w["technical"] += deficit * 0.7
        w["sentiment"] += deficit * 0.3
        state.skip_fundamental = True
        state.log("Orchestrator: no financials → skip fundamental, boost
technical")

    # Normalise so weights always sum to 1.0
    total = sum(w.values())
    state.weights = {k: round(v / total, 3) for k, v in w.items()}

```

```

        return state

    def _empty_output(self, agent_name: str) -> dict:
        """Return an empty output for a skipped agent."""
        return {"agent_name": agent_name, "status": "skipped", "confidence":
0.0, "payload": {}}
```

File: tests/test_sprint1.py

```

# tests/test_sprint1.py
# Run this at the end of Sprint 1 to verify everything works.
# Command: python tests/test_sprint1.py

import asyncio
import asyncpg
import os
from dotenv import load_dotenv
from agents.agent_state import AgentState
from agents.orchestrator import OrchestratorAgent

load_dotenv()

async def test_database_connection():
    """Test 1: Can we connect to PostgreSQL?"""
    print("\nTest 1: Database connection...")
    conn = await asyncpg.connect(
        host=os.getenv("DB_HOST", "localhost"),
        port=int(os.getenv("DB_PORT", 5432)),
        database=os.getenv("DB_NAME", "bvmt_db"),
        user=os.getenv("DB_USER", "postgres"),
        password=os.getenv("DB_PASSWORD", "")
    )
    result = await conn.fetchval("SELECT COUNT(*) FROM daily_prices")
    await conn.close()
    print(f" PASS: {result} rows in daily_prices")
    assert result > 0, "daily_prices table is empty!"

async def test_five_stocks_loaded():
    """Test 2: Are all 5 test stocks in the database?"""
    print("\nTest 2: 5 stocks loaded...")
    conn = await asyncpg.connect(
        host=os.getenv("DB_HOST", "localhost"),
        port=int(os.getenv("DB_PORT", 5432)),
        database=os.getenv("DB_NAME", "bvmt_db"),
        user=os.getenv("DB_USER", "postgres"),
        password=os.getenv("DB_PASSWORD", "")
    )
    rows = await conn.fetch("SELECT DISTINCT ticker FROM daily_prices ORDER BY
ticker")
    await conn.close()
    tickers = [r["ticker"] for r in rows]
    print(f" Stocks found: {tickers}")
    assert len(tickers) >= 5, f"Expected 5 stocks, found {len(tickers)}"
    print(" PASS")
```

```

async def test_agent_state():
    """Test 3: Does AgentState initialise correctly?"""
    print("\nTest 3: AgentState...")
    state = AgentState(ticker="BIAT", user_role="manager")
    assert state.ticker == "BIAT"
    assert state.weights["technical"] == 0.42
    assert state.skip_sentiment == False
    print(f" PASS: request_id = {state.request_id[:8]}...")

async def test_weight_adjustment():
    """Test 4: Does Orchestrator adjust weights correctly for low news?"""
    print("\nTest 4: Weight adjustment...")
    orch = OrchestratorAgent()
    state = AgentState(ticker="TEST", user_role="manager")
    state.news_articles_count = 2 # very low news
    state.has_recent_financials = True
    state = orch._adjust_weights(state)
    print(f" Adjusted weights: {state.weights}")
    assert state.skip_sentiment == True, "Should skip sentiment with 2
articles"
    assert state.weights["technical"] > 0.42, "Technical weight should
increase"
    total = sum(state.weights.values())
    assert abs(total - 1.0) < 0.001, f"Weights must sum to 1.0, got {total}"
    print(" PASS")

async def test_full_orchestrator():
    """Test 5: Full Orchestrator run for BIAT."""
    print("\nTest 5: Full Orchestrator run for BIAT...")
    orch = OrchestratorAgent()
    result = await orch.run("BIAT", "manager")
    print(f" Result: {result}")
    assert result["ticker"] == "BIAT"
    assert result["data_quality"] >= 0.0
    print(" PASS")

async def main():
    print("=" * 50)
    print("SPRINT 1 INTEGRATION TESTS")
    print("=" * 50)
    await test_database_connection()
    await test_five_stocks_loaded()
    await test_agent_state()
    await test_weight_adjustment()
    await test_full_orchestrator()
    print("\n" + "=" * 50)
    print("ALL TESTS PASSED - Sprint 1 complete!")
    print("=" * 50)

asyncio.run(main())

```

File: .env (create in project root — never commit to GitHub!)

```
# .env - database connection settings
# IMPORTANT: add .env to your .gitignore file!

DB_HOST=localhost
DB_PORT=5432
DB_NAME=bvmt_db
DB_USER=postgres
DB_PASSWORD=your_pgadmin_password_here
```

9.4 How to Verify Each Step Works

Step	Command to Run	What You Should See
Python installed	python --version	Python 3.11.x
Virtual env active	where python (Windows)	Path inside your venv folder
Libraries installed	python -c "import pandas; print('OK')"	OK
PostgreSQL connection	python -c "import asyncpg; import asyncio; asyncio.run(asyncpg.connect(host='localhost',port=5432,database='bvmt_db',user='postgres',password='YOUR_PW'))"	No error
Tables created	SELECT COUNT(*) FROM information_schema.tables WHERE table_schema='public' (in pgAdmin)	11
Data loaded	SELECT ticker, COUNT(*) FROM daily_prices GROUP BY ticker (in pgAdmin)	5 rows with counts
Sprint 1 tests	python tests/test_sprint1.py	ALL TESTS PASSED

9.5 Using Lightning AI for Training (Sprint 2 Onwards)

You do not need Lightning AI until Sprint 2 (Week 3). But prepare it now.

Sprint 2 you will upload your CSV data to Lightning AI and run TFT training there.

The trained model file (.pt) comes back to your local /models/ folder.

All inference (using the model to make predictions) runs locally — no GPU needed for that.

Workflow for Sprint 2+ Training

6. Upload your CSV files: In Lightning AI Studio, drag data/prices/*.csv into the file browser.
7. Create training/train_tft.py locally (the CDC gives you the training code). Upload it to Lightning AI.
8. In Lightning AI terminal: pip install pytorch-forecasting torch pandas pandas-ta
9. Run: python train_tft.py — this uses the free GPU and takes 20–40 minutes.
10. Download the trained model: right-click the .pt file in Lightning AI file browser → Download → save to your local models/ folder.
11. Back on your Windows machine: your agents will load the .pt file locally for inference — no GPU needed.

10. The 8 Most Common Confusions — Cleared Up

Q1: Is an Agent an AI?

No. An agent is just a Python class with a `run()` method. `DataAgent` is pure ETL. `RiskAgent` is statistics. Only `TechnicalAgent` and `SentimentAgent` run actual AI models inside them.

Q2: What is `asyncio.gather()`?

It means: run multiple things at the same time and wait for all to finish. Like opening 3 Excel files simultaneously. In this project it runs `TechnicalAgent`, `FundamentalAgent`, and `SentimentAgent` in parallel — cutting wait time by 60%.

Q3: What is a dataclass in Python?

A dataclass is just a Python class where you only define the variables — no complicated methods. `AgentState` is a dataclass: it holds ticker, data quality, weights, results.

Q4: I had Docker in the old version — do I need it now?

No. You have PostgreSQL and pgAdmin already installed. Docker was just a way to run PostgreSQL inside a container. Since you already have it natively on Windows, you skip Docker entirely. This is actually simpler.

Q5: What is the difference between TFT and CNN-LSTM?

Both predict time series. CNN-LSTM is older and simpler — used as a baseline to compare against. TFT is newer and better — the primary model. You train both, compare accuracy, and the difference is one academic contribution.

Q6: What is Redis?

A very fast temporary cache. When the system computes a full analysis for BIAT, it stores the result for 4 hours. The next request gets it instantly. Think of it as a materialised view that refreshes every 4 hours. Install it locally: download redis from github.com/microsoftarchive/redis/releases

Q7: What is walk-forward cross-validation?

A way of testing AI models without cheating. Train on 2015–2018, test on 2019. Train on 2015–2019, test on 2020. Etc. You never let the model see future data during training.

Q8: What is SHAP in simple terms?

SHAP answers: which inputs were most responsible for this prediction? It is like a decomposition waterfall chart — instead of just showing the total KPI, you show how much each component contributed.

11. Your Personal Learning Path

Priority	Skill	How Hard?	Learn When
CRITICAL	Python — pandas, functions, classes, loops	Easy if you know Excel formulas	Week 1
CRITICAL	SQL — SELECT, JOIN, INSERT, GROUP BY	You know this from BI — review only	Week 1
CRITICAL	pgAdmin — creating databases, running queries	You already know this	Day 2
HIGH	PostgreSQL specifics — asyncpg async driver	Small extension of SQL you know	Week 2
HIGH	pandas_ta — compute technical indicators	Just function calls, very easy	Week 3
HIGH	HuggingFace — load and run a pre-trained model	5 lines of code to start	Week 5
MEDIUM	Lightning AI — upload data, run training script	Simple file upload + run command	Week 3
MEDIUM	pytorch-forecasting — TFT training	Follow tutorials, no math needed	Week 3–4
MEDIUM	XGBoost — train and use	Very well documented, easy to follow	Week 5
MEDIUM	torch_geometric — GNN	Harder — many tutorials exist	Week 7
LOW	MCP protocol — wrap agents in MCP	Can do LAST — agents work without it	Week 9
LOW	asyncio — parallel execution	Learn ONE example and copy pattern	Week 5–6

The most important advice in this document

You do not need to understand EVERYTHING before you start.
You need to understand the NEXT STEP only.

On Day 1, you only need to know: how to install Python and run a script.
On Day 3, you only need to know: how to read a CSV with pandas.
On Day 5, you only need to know: how to connect to PostgreSQL.

Start. Build. Break things. Fix them. Understand as you go.